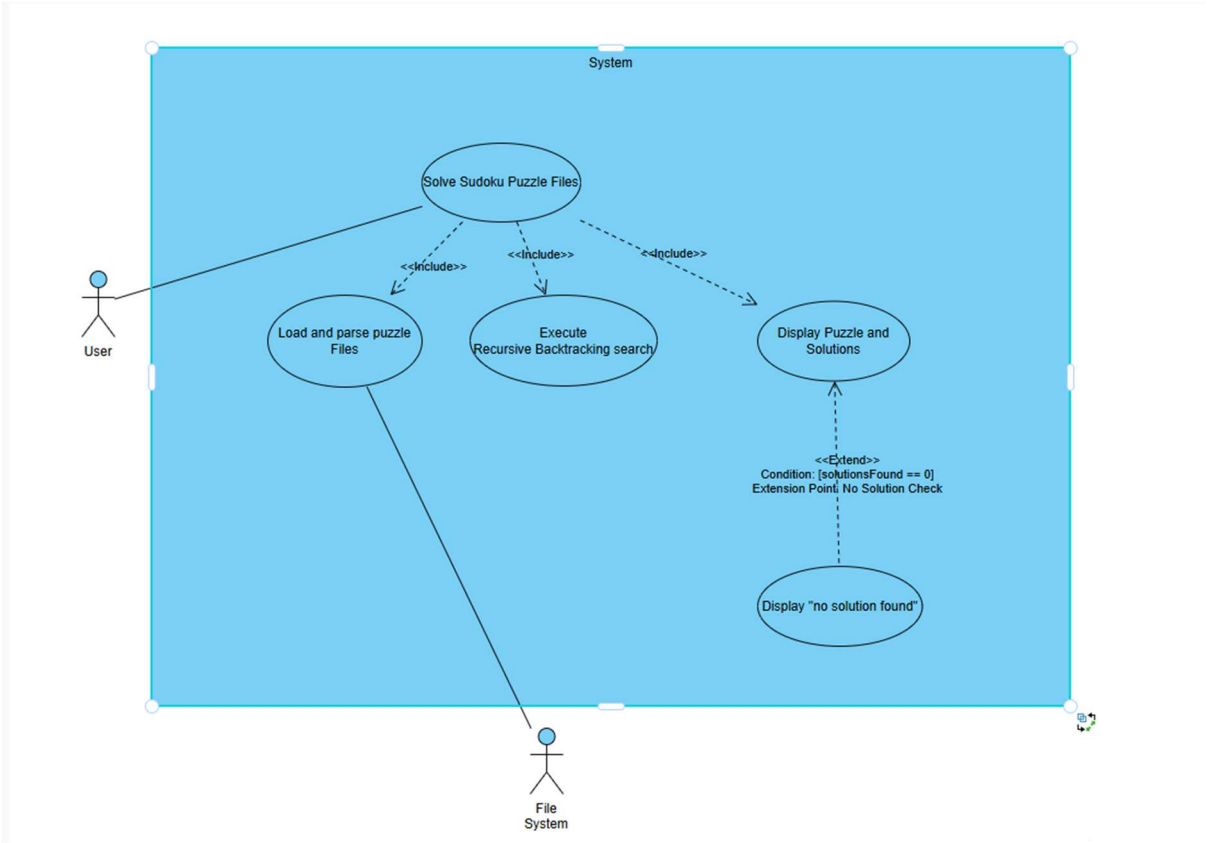


1) Identify a Use Case

- **Use Case Name:** Solve Sudoku Puzzle Files (UC-01)
- **Primary Actor:** User (Student / GTA executing ./Assignment4.exe)
- **Secondary Actor:** File System (supplying puzzle1.txt through puzzle5.txt)

- **Included Use Cases (**<<include>>**):** Load and parse puzzle Files, Execute Recursive Backtracking search, and Display Puzzle and Solutions

- **Extending Use Case (**<<extend>>**):** Display "no solution found" (triggered on condition [solutionsFound == 0])

2) Brief Description for the Use Case

The User runs Assignment4.exe to sequentially load five 9×9 Sudoku puzzle files (puzzle1.txt to puzzle5.txt), print each puzzle's filename and initial board configuration, recursively search for all valid solutions using depth-first search with backtracking, and print either every valid solved grid or "No solution found" to standard output.

3) Step-by-Step Flow (Outline) of Events

1. The User launches ./Assignment4.exe from the command line.

2. The system instantiates a SudokuSolver object and initializes the file list (puzzle1.txt through puzzle5.txt).

3. For each file in the list:

- **a.** The system outputs the puzzle filename.
- **b.** The system reads the 81 tokens from the file ('_' as 0, '1'–'9' as integers 1–9).
- **c.** The system prints the initial unsolved grid to the console.
- **d.** The system validates initial clues and starts recursive backtracking at cell (0, 0).
- **e.** For each empty cell, the system tests digits 1–9 against row, column, and 3×3 block constraints (isSafe).
- **f.** Upon reaching row == 9, the system increments the solution counter, prints the completed grid, and backtracks to search for additional solutions.
- **g.** If the search finishes with zero solutions found, the system outputs "No solution found".

4. After processing all five files, the program terminates with exit status 0.

4) Detailed Description for the Use Case

- **Use Case ID & Name:** UC-01: Solve Sudoku Puzzle Files

- **Actors:** User (Primary), File System (Secondary)

- **Preconditions:**

    1. Assignment4.cpp is compiled into Assignment4.exe on the EECS Cycle server.

    2. Input files puzzle1.txt through puzzle5.txt exist in the working directory.

- **Trigger:** The User runs ./Assignment4.exe in the terminal.

- **Main Success Scenario (Basic Flow):**

    1. The User runs ./Assignment4.exe.

    2. The system constructs SudokuSolver with a zeroed 9×9 grid and solutionsFound = 0.

    3. The system loops through files puzzle1.txt through puzzle5.txt.

    4. For each file, the system prints the filename banner, opens the file, and loads 81 tokens into the 2D array.

    5. The system calls printGrid() to display the unsolved initial grid.

6. The system verifies initial clue validity and invokes backtrack(0, 0).

7. In backtrack(row, col):

    - If the cell is pre-filled, recursion advances to (nextRow, nextCol).

    - If the cell is empty (0), digits 1–9 are tested with isSafe(row, col, num).

    - For each valid digit, the digit is assigned to grid[row][col] and backtrack(nextRow, nextCol) is called.

    - When row == 9 is reached, the system increments solutionsFound, prints "Solution X:", and prints the solved grid.

    - Backtracking resets grid[row][col] = 0 to explore remaining branches.

8. Once all five files are processed, the program exits cleanly with code 0.

- **Alternative Flows:**

    - **Unsolvable Puzzle (**puzzle4.txt**):** If all paths encounter conflicts and backtrack(0, 0) ends with solutionsFound == 0, the system outputs "No solution found".

    - **Multiple Solutions:** After printing a solution at row == 9, recursion returns to explore remaining candidates, printing every valid solution found.

    - **File Read Error:** If a file fails to open, the system outputs an error message and proceeds to the next file.

- **Postconditions:** The filename, initial puzzle, and all solutions (or "No solution found") are printed to standard output for all five files, and all file handles are closed.